



Operating Systems Lab

Course Code: CSE-412

Experiment Title: First Come First Serve (FCFS) CPU Scheduling Algorithm

Student Name: Md. Shoyaif Rahman

ID: 0432310005101050

Instructor Name: Afroja Ahmed Smrity

University Name: University of Information Technology and Sciences (UITS)

Experiment No: 02

Experiment Title: Implementation of FCFS Scheduling

Date of Performance: 10/02/26

Date of Submission: 24/02/26

Objective of the Experiment

- To understand the FCFS CPU scheduling algorithm.
- To implement the FCFS algorithm in C++.
- To calculate Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process.
- To compute the average WT and average TAT.

Theory / Background

First Come, First Served (FCFS) is the simplest CPU scheduling algorithm. The process that requests the CPU first is allocated the CPU first. It is implemented using a FIFO (First In, First Out) queue. FCFS is non-preemptive, meaning once a process gets the CPU, it holds it until it terminates or performs I/O.

- **Waiting Time (WT):** Time a process spends waiting in the ready queue. For the first process, $WT = 0$. For subsequent processes, $WT[i] = WT[i-1] + BT[i-1]$.
- **Turnaround Time (TAT):** Total time from process arrival to completion. $TAT[i] = WT[i] + BT[i]$.
- **Completion Time (CT):** Time at which the process finishes execution. $CT[i] = TAT[i] + AT[i]$ (if arrival time is 0, $CT = TAT$).

Procedure

1. Start the program.
2. Input the number of processes (n).
3. Input the Burst Time (BT) for each process.
4. Set the Waiting Time of the first process to 0.
5. Calculate the Waiting Time for subsequent processes using the formula: $WT[i] = WT[i-1] + BT[i-1]$.
6. Calculate the Turnaround Time for each process: $TAT[i] = WT[i] + BT[i]$.
7. Calculate the average WT and average TAT.
8. Display the results in a table format.
9. End the program.

Program Code:

```
#include <iostream>
using namespace std;
```

```

int main() {
    int n;
    float avgWT = 0, avgTAT = 0;
    cout << "Enter number of processes: ";
    cin >> n;
    int bt[n], wt[n], tat[n];
    cout << "Enter burst time for each process:" << endl;
    for (int i = 0; i < n; i++) {
        cout << "Process P" << i + 1 << ": ";
        cin >> bt[i];
    }

    wt[0] = 0;
    for (int i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    cout << "\nProcess\tBT\tWT\tTAT\n";
    for (int i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
        avgWT += wt[i];
        avgTAT += tat[i];
        cout << "P" << i+1 << "\t" << bt[i] << "\t" << wt[i] << "\t" << tat[i] << endl;
    }
    cout << "\nAverage Waiting Time: " << avgWT/n << endl;
    cout << "Average Turnaround Time: " << avgTAT/n << endl;
    return 0;
}

```

Input and Output

Input:

Enter number of processes: 3

Enter burst time for each process:

Process P1: 24

Process P2: 3

Process P3: 3

Output:

Process	BT	WT	TAT
P1	24	0	24
P2	3	24	27
P3	3	27	30
Average Waiting Time: 17			
Average Turnaround Time: 27			

Analysis / Discussion

The FCFS algorithm is simple and easy to implement. However, it suffers from the convoy effect, where short processes get stuck behind a long process, leading to high average waiting times. As seen in the example, P2 and P3 (with BT=3) had to wait for P1 (BT=24) to finish. This makes FCFS unsuitable for time-sharing systems. The algorithm favors CPU-bound processes over I/O-bound processes.

Conclusion

The FCFS scheduling algorithm was successfully implemented. While straightforward, its performance is often poor due to high waiting times for short processes, a phenomenon known as the convoy effect. This experiment highlights the need for more efficient algorithms like SJF or Round Robin for interactive and multitasking environments.